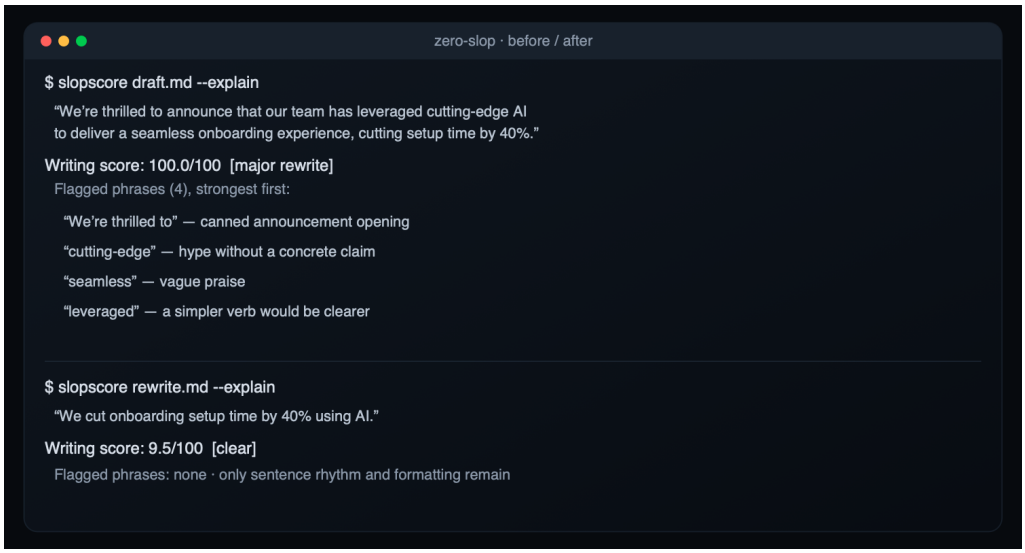
Zero Slop

You used AI to help with a draft, but the result sounds generic. You can hear it, and so can everyone who reads it. That machine sound has a name: slop.

Zero Slop is not an AI model. It runs inside the AI assistant you already use. Claude, GPT, or another compatible model reads and edits the draft; Zero Slop supplies the method and local checks. Its writing score runs from 0 to 100 and points to the phrases and writing patterns that raised it. The assistant then runs separate copy-editing and read-aloud passes. Final checks make sure the facts survived.

```
npx skills add manavmishra/ZeroSlop --global
```

Then open your assistant and say "de-slop this." The Python scorer needs no account or server and does not transmit your writing.



Why it matters now

Sounding like AI can cost credibility, and platforms are adding ways to report, classify, or limit repetitive machine-made material. LinkedIn added an AI-slop report signal; Snap and YouTube apply recommendation or monetization rules to some synthetic or mass-produced content; Substack lets readers scan text for AI signals. Those are different policies, not proof of one universal reach penalty. The practical point is simpler: readers notice generic machine prose, and writers deserve a way to inspect and fix it before publishing.

Who it is for

Anyone who writes under their own name and would rather not sound like a robot. A founder writing a launch post, a marketer shipping five things a week, a researcher who needs their paper to stay formal, an engineering team that wants slop to fail a check the way a bug does.

Seven roles, one workflow

The local scorer finds exact phrases and problems with rhythm, readability, and formatting. The AI assistant acts as interpreter, then rewriter. The local fact gate rejects versions that add or lose names, numbers, quotations, or links. Fresh AI passes act as the copy desk, then the read-aloud editor. Finally, local tools and the assistant share the verifier role, comparing the exact finished text with the source. These are seven roles, not seven models. Separating them stops the rewriter from certifying its own work and keeps late edits from bypassing the final checks.

What makes it different

It builds on five open-source projects that worked out this craft first: no-ai-slop, humanizer, de-slop, stop-slop, and unslop-text. It adds three things a plain rewrite lacks.

A score you can inspect. The rules and structural measures are visible, and the same threshold check can run in a build. It is a repeatable writing check, not authorship proof.

A promise about your facts. A rewrite can invent a detail or drop a number to make a sentence flow. Zero Slop lists every figure, name, quote, and link in your original and rejects a version that loses one or adds one. A missing number may be obvious; an invented one can read naturally enough to go unnoticed, which is why the check exists.

A tool that learns from later edits. The editorial loop handles the current draft. A separate private learning loop can compare the assistant's version with a later edit: a phrase you cut may reveal a pattern the local check missed, while flagged text you keep may be a false alarm. No detection rule or preferred fix becomes active after one edit. Before a phrase rule can take effect, the same cut must appear in three unrelated pieces. It must also be new and leave a reference set of human writing unflagged; a single-word cut must appear in five unrelated pieces. A preferred replacement must also recur in three unrelated pieces. The private learning file loads on the next run. Later matching edits confirm useful rules and fixes; without reconfirmation, stale detection rules decay and stale preferred fixes retire.

This is post-deployment, human-in-the-loop online learning. It adapts detection and fixing through inspectable local rules and preferences. Human corrections provide feedback, but Zero Slop does not perform reinforcement learning or RLHF, nor does it retrain the AI model already running in the assistant. Shared changes still require review, tests, versioning, and a release. Each session can check for a newer release with a metadata-only version query. That query never sends the draft.

What we tested

Version 2.5.5 reran the complete test suite, scores, external corpus audits, speed record, and charts; v2.5.6 added the performed-writer tell family. Version 2.5.7 put context before isolated vocabulary and fixed machine-readable batch output. Version 2.5.8 adds one conservative generic-benefit cluster. Its faster scanning engine matched v2.5.7 on all 280 tracked documents; only the new rule changes a score. The main check covers all 8,000 rows in the MIT-licensed RAID+ dataset. After excluding 373 failed or empty outputs, 7,627 abstracts remained. Mean writing scores ranged from 14.5 for DeepSeek V3 to 25.5 for Llama 3.3 70B. RAID+ labels model origin, not editorial quality, so this is a current-model distribution check, not an accuracy claim.

A fresh pass over all 2,187 Beemo records found mean scores of 30.2 for raw model responses, 25.3 for expert edits, and 20.0 for independent human answers. Expert editing lowered the score in 52.2 percent of pairs. Beemo also lacks writing-quality labels. On a frozen 38-passage consensus panel, the writing gate's accuracy rose from 71.1 percent to 84.2 percent, correcting five false negatives without changing a clean consensus item. Two LLM editorial raters supplied those labels, so this is an internal regression result, not human field accuracy. The current 1,000-document speed run took 2.0147 seconds on one Apple silicon Mac; an alternating comparison measured 29.4 percent higher median throughput than v2.5.7. These are reproducible checks with stated limits, not universal claims about writing quality.

Open source under the MIT License · github.com/manavmishra/ZeroSlop · v2.5.8 · tested · built on no-ai-slop, humanizer, de-slop, stop-slop, and unslop-text, with thanks to Kagi's SlopStop and the research listed in the repo.